# DLL Developer Hub Prototype - Project Specification (Revised)

HCI 584 - Python Application Development in HCI - Andrea Bierstedt - Fall 2026

## How revisions are marked

This is my original spec with Version 1 review changes added. Changes are marked like this:

- **[REVISED]** - something in the original spec that changed
- **[ADDED]** - something new that is already built in Version 1
- **[MOVED TO V2]** - planned for Version 1 but now part of Version 2
- **[DROPPED]** - removed from the project
- **[PROPOSED - nice to have]** - an extra feature I'll try only if time permits

Everything not marked is unchanged from the original spec.

## General Description of the Project

The DLL Developer Hub Prototype will be a small web application that gives developers one starting place for learning about available finance APIs and requesting access to use them. The home page will begin with two main areas: API Catalog and Request Credentials. These two choices represent the main activities I want to explore in the project. A user can either learn what APIs are available or begin the process of getting access to an API.

The API Catalog will let the user browse a small collection of API products. The products will be organized around understandable goals, such as creating a quote, submitting a finance application, or viewing contract information. A user can open a product to read a short overview, see when it should be used, review basic requirements, and look at a simplified API specification. The specification may include operation names, paths, required information, and sample request and response data.

The Request Credentials area will guide the user through a basic access request. The user will select an API, choose an environment, provide simple developer and organization information, and answer a few readiness questions. The app will check whether the required information is complete and show the request status. For the class project, credentials and approvals will be simulated rather than connected to a real credential system.

Version 1 can run as a command-line application using print() and input(). The application data can be stored in local JSON files and loaded into Python lists and dictionaries. Version 2 will ideally use Streamlit so the project can have a home page, two large navigation choices, catalog cards, forms, buttons, and status messages. The application will not need to offer its own API, although that could be considered later if the main project is completed.

**[REVISED]** Version 1 (command line) is complete. Version 2 **will** use Streamlit (no longer "ideally"). The Streamlit app will be a single `app.py` at the project root that imports the logic functions from `main.py`. Navigation will use `st.session_state` instead of a `pages/` folder, so all `.py` files stay at the project root.

**[DROPPED]** Offering the app's own API. The remaining time will go into the Streamlit version.

## Task Vignettes (User Activity Flow)

### 1) Start from the home page

Jordan is a developer who needs to add financing features to an equipment dealer application. Jordan opens the Developer Hub and sees a simple home page with two main choices: API Catalog and Request Credentials. Jordan is still researching the available services, so Jordan selects API Catalog.

Technical details and ideas for later:

- Version 1 will show the same two choices as a numbered command-line menu. **[REVISED]** Done; the menu also has a third choice, Exit.
- Version 2 will show two large cards or buttons on the Streamlit home page.
- The selected choice will determine which part of the program runs next.
- A Back or Home choice will return the user to the starting page. **[REVISED]** In Version 1, every flow returns to the home menu automatically. Version 2 will add Back/Home buttons.

### 2) Browse the API catalog and view an API

The catalog displays several finance-related API products. Jordan can browse the list or search for a word such as quote. Jordan selects a Quote API product and reads a short explanation of what it does. The page also shows common uses, basic requirements, related APIs, and the environments in which the API is available.

Jordan selects View API Specification. The application shows a simplified list of operations. Jordan opens one operation and sees the method, path, required values, sample request, and sample response. Jordan can return to the product overview, choose another operation, or begin a credential request for the selected API.

Technical details and ideas for later:

- Catalog and specification information will be stored in local JSON files.
- Python will load the data into lists and dictionaries.
- Search will compare the user's text with product names, descriptions, and keywords.
- Each API product will have an ID that connects it to its specification information.
- Version 1 will print the product and specification information in the terminal.
- Version 2 may use cards for products and tabs for Overview and API Specification.
- **[ADDED]** The catalog has 3 products with 2 operations each. The user chooses which operation's spec to view from a numbered list.
- **[MOVED TO V2]** Showing related APIs on the product page. This needs a new "related" field in `catalog.json`.
- **[MOVED TO V2]** Beginning a credential request directly from the product page. In Version 1 the user goes back to the home menu and chooses Request Credentials.

### 3) Request credentials

After reviewing the Quote API, Jordan selects Request Credentials. The application asks Jordan to choose the API and environment. Jordan then enters a name, organization, email address, and a short

description of how the API will be used. The application also asks a few yes-or-no questions, such as whether the API guide has been reviewed and whether a technical contact is available.

Jordan submits the request. The application checks the required fields and displays either a list of missing information or a confirmation message with a request number and status. For the class prototype, the status may change from Submitted to Approved through a simulated review step. When approved, the application displays sample credential information with the key hidden by default.

Technical details and ideas for later:

- The request form will create a dictionary containing the user, API, environment, intended use, and readiness answers.
- A validation function will return a list of missing fields or requirements.
- Requests will be saved in a local JSON file so they can be viewed again.
- Request statuses may include Draft, Submitted, Needs Information, Approved, and Credentials Available.
- The sample credential will be connected to the approved request by a request ID.
- Version 2 may show the request status and masked credential on a simple dashboard.
- **[ADDED]** The user also selects one or more API operations (e.g. "1,2") as part of the request.
- **[ADDED]** Validation also rejects API ids that aren't in the catalog, environments other than sandbox/production, and emails without "@".
- **[REVISED]** Statuses are now **Submitted → Credentials Available** or **Submitted → Needs Information**. "Draft" is dropped because requests are only saved once they pass validation. "Approved" and "Credentials Available" are combined because the sample key is issued at approval.
- **[REVISED]** The simulated review: a request goes to Credentials Available if the user answered "yes" to both readiness questions. Otherwise it goes to Needs Information with a reviewer note. In Version 1 the user runs the review from "View existing request".

### 4) Return and check the request

Jordan later returns to the Developer Hub and chooses Request Credentials from the home page. The application offers the choice to start a new request or view an existing request. Jordan enters the request number and sees the selected API, environment, current status, and any remaining steps. If the request is approved, Jordan can view the sample credential and return to the API specification for authentication instructions.

Technical details and ideas for later:

- The program will search the saved request list by request ID.
- The displayed result will combine request, API, and credential information.
- The credential value will be masked unless the user chooses to reveal it. **[MOVED TO V2]** The masking is done in Version 1, but the "reveal" choice will be a toggle in Streamlit.
- A future improvement could add separate validation and production credential requests. **[PROPOSED - nice to have]**
- **[ADDED]** Request IDs are not case-sensitive when looked up (req-abc123 finds REQ-ABC123).

- **[DROPPED]** "Return to the API specification for authentication instructions". The simplified specs don't include authentication details.
- **[PROPOSED - nice to have]** Let the user fix and resubmit a "Needs Information" request.

## Technical Flow

The project will have one main menu that represents the home page. From the home page, the program will send the user to either the catalog flow or the credential-request flow. Both areas will use shared API product data so that the selected API remains consistent throughout the experience.

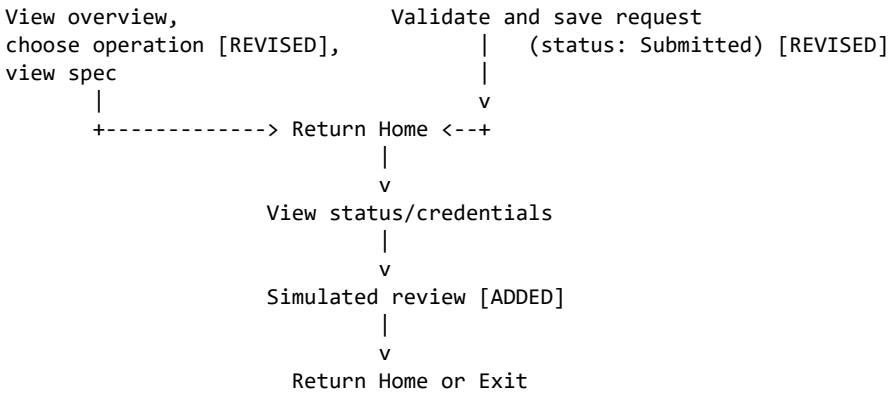
### High-level data flow

| Step | Program block | Input | Output |
|---|---|---|---|
| 1 | Load data | Catalog, specification, and request JSON files **[REVISED]** | Python lists and dictionaries |
| 2 | Show home menu | Two main choices | API Catalog or Request Credentials selection |
| 3A | Browse catalog | Search text or product selection | Matching API products and product details |
| 4A | Show specification | Product ID and operation selection | API operation and sample data |
| 3B | Collect credential request | User, API, operations **[ADDED]**, environment, and readiness answers | Request dictionary |
| 4B | Validate and save request | Request dictionary | Missing items or saved request with status "Submitted" **[REVISED]** |
| 5B | View request and credentials | Request ID | Request status and sample credential information |
| 5C **[ADDED]** | Simulated review | Submitted request | "Credentials Available" with sample key, or "Needs Information" with a note |
| 6 | Return home or exit | User choice | Main menu or end of program |

**[REVISED]** Step 1: there is no separate credentials JSON file. The sample credential is stored on the request in `requests.json`.

### Simple flow diagram

```
Start application
       |
       v
Load JSON data
       |
       v
Home page / main menu
       |
       +------------------------------+
       |                              |
       v                              v
API Catalog                    Request Credentials
       |                              |
Search or browse               New request or existing request
       |                              |
Select API                     Enter request information
       |                              |
```




## Main functions I expect to need

- load_data() to read the JSON files.
- show_home_menu() to display the two main areas.
- search_catalog() to find matching API products.
- show_api_product() to display the product overview.
- show_api_spec() to display one API operation.
- collect_credential_request() to gather the form answers.
- validate_request() to identify missing information.
- save_request() to store the request in JSON.
- find_request() to retrieve a saved request by ID.
- show_credentials() to display masked sample credentials.
- main() to control the overall program loop.

**[ADDED]** Functions added during Version 1:

- find_product() to check that an API id exists in the catalog.
- list_operations() and choose_operations() to show and select API operations.
- write_requests() to save the request list to JSON.
- review_request() to run the simulated review.
- run_catalog_flow() and run_credential_flow() to run each of the two main areas.

**[REVISED]** For Version 2, functions that print will be split so the logic returns data and the display is handled separately. That way the Streamlit `app.py` can reuse the same logic.

## Somewhat unknown

- How many API products and operations are needed to make the catalog feel useful? **[REVISED - answered]** 3 products with 2 operations each is enough to demo search and operation selection. More can be added to the JSON files without code changes.
- Should the API overview and specification appear on one page or separate pages? **[REVISED - answered]** One product page with two tabs (Overview, API Specification) in Version 2.

- How detailed should the credential-readiness questions be? **[REVISED - answered]** Two yes/no questions (guide reviewed, technical contact) are enough to drive the simulated review.
- How should a simulated approval step work in Version 1? **[REVISED - answered]** See request statuses in vignette 3.
- Will there be enough time to add separate validation and production credential requests? **[REVISED]** Moved to nice-to-have.

## Version 2 plan [ADDED]

Version 2 is a Streamlit app (`app.py`, run with `streamlit run app.py`):

1. **Home page:** two large buttons or cards - API Catalog and Request Credentials.
2. **API Catalog page:** search box, product cards, and a product page with Overview and API Specification tabs.
3. **Request Credentials page:** a form (`st.form`) with validation messages and a confirmation showing the request ID.
4. **Request status page:** look up by request ID, run the simulated review, and show a masked key with a reveal toggle.

**[PROPOSED - nice to have]** features, only if time permits:

- "Request credentials for this API" button on the product page that pre-fills the form
- Related APIs on the product page
- Fix and resubmit a "Needs Information" request
- Separate sandbox and production credential requests